Aarya Arban

T11 – 05

Assignment No. 2

Aim: Data Preparation using Pandas

Theory:

a. What is data cleaning and why you need it.

Data cleaning (also known as data cleansing or data scrubbing) is the process of detecting, correcting, or removing inaccurate, incomplete, inconsistent, or duplicated data from a dataset. It ensures that the data is accurate, reliable, and ready for analysis.

Why Do You Need Data Cleaning?

Data cleaning is essential because:

1. **Improves Data Quality** – Ensures accurate and consistent data for better decision-making.
2. **Enhances Model Performance** – Clean data leads to better predictions in machine learning and analytics.
3. **Reduces Errors** – Eliminates inconsistencies and missing values that can cause incorrect results.
4. **Increases Efficiency** – Saves time and resources by preventing data issues in later stages.
5. **Prevents Misinterpretation** – Avoids misleading conclusions due to incorrect or duplicate data.

b. What are the various techniques of data cleaning?

Techniques of Data Cleaning

1. **Removing Duplicate Data**
    - Identifying and eliminating redundant records that appear multiple times.
2. **Handling Missing Values**
    - **Removal**: Deleting rows/columns with excessive missing values.
    - **Imputation**: Filling missing values using mean, median, mode, or predictive modeling.
3. **Standardizing Data**
    - Ensuring consistency in data formats (e.g., date formats, text capitalization, address formats).
4. **Correcting Inconsistencies**
    - Fixing incorrect spellings, abbreviations, typos, and inconsistent naming conventions.
5. **Outlier Detection and Removal**
    - Identifying and handling extreme values using statistical methods like Z-score, IQR, or visualization techniques.
6. **Validating Data Accuracy**
    - Cross-checking data with reference datasets or using validation rules to ensure correctness.
7. **Removing Irrelevant Data**
    - Eliminating unnecessary columns or records that do not add value to the analysis.
8. **Fixing Data Type Issues**
    - Converting data types (e.g., strings to dates, integers to floats) for proper processing.
9. **Ensuring Consistent Units & Measurements**
    - Standardizing measurement units (e.g., converting all weights to kilograms or all prices to USD).
10. **Text Data Cleaning** (for NLP or textual analysis)

- Removing special characters, stop words, and performing tokenization and stemming.

Data cleaning is a crucial step in **data preprocessing** to enhance the quality, reliability, and accuracy of data for analysis or machine learning models.

Data Preparation Using Pandas

1. Importing Data

Pandas allows the easy import of data from various file formats such as CSV, Excel, JSON, and databases. Properly loading data is the first step in the preparation process.

2. Handling Missing Values

Data often contains missing or null values, which can be managed by either dropping them or filling them with appropriate values, such as the mean, median, or mode of the column.

3. Handling Duplicates

Duplicates in data can distort analysis. Identifying and removing duplicate rows ensures the data remains accurate and clean.

4. Data Type Conversion

Ensuring that data is in the correct format is crucial for analysis. Converting data types allows for proper mathematical or logical operations on the data.

5. Feature Scaling

When features in the dataset have different scales, normalization or standardization is applied to bring them to a common scale. This step is essential for certain algorithms, especially in machine learning.

6. Encoding Categorical Data

Machine learning algorithms generally require numerical inputs. Therefore, categorical data needs to be encoded into numerical values, often using techniques like one-hot encoding or label encoding.

7. Splitting Data

Before applying any models or performing analysis, it's common to split the dataset into training and testing sets to evaluate performance accurately without overfitting.

These data preparation steps help ensure the dataset is ready for analysis or machine learning tasks, providing reliable and accurate results.

Output:

1. Find the null values in each column

```
null_count_1 = df1.isnull().sum()
print(null_count_1)
```

[18] ✓ 0.0s

```
match_id                0
inning                  0
batting_team            0
bowling_team            0
over                    0
ball                    0
batter                  0
bowler                  0
non_striker             0
batsman_runs            0
extra_runs              0
total_runs              0
extras_type        246795
is_wicket               0
player_dismissed   247970
dismissal_kind     247970
fielder            251566
dtype: int64
```

2. Merge the two datasets

```
merged_df = pd.merge(df1, df2, on='match_id', how='left')
```
[25] ✓ 0.2s Python

```
merged_df.head()
```
[26] ✓ 0.0s Python

iker	batsman_runs	...	toss_decision	winner	result	result_margin	target_runs	target_overs	super_over	method	umpire1	umpire2
BB llum	0	...	field	Kolkata Knight Riders	runs	140.0	223.0	20.0	N	NaN	Asad Rauf	RE Koertzen
guly	0	...	field	Kolkata Knight Riders	runs	140.0	223.0	20.0	N	NaN	Asad Rauf	RE Koertzen
guly	0	...	field	Kolkata Knight Riders	runs	140.0	223.0	20.0	N	NaN	Asad Rauf	RE Koertzen
guly	0	...	field	Kolkata Knight Riders	runs	140.0	223.0	20.0	N	NaN	Asad Rauf	RE Koertzen
guly	0	...	field	Kolkata Knight Riders	runs	140.0	223.0	20.0	N	NaN	Asad Rauf	RE Koertzen

3. Find the null values in each column again

```
null_count_3 = df.isnull().sum()
print(null_count_3)
```
[27] ✓ 0.0s

```
match_id              0
inning                0
batting_team          0
bowling_team          0
over                  0
ball                  0
batter                0
bowler                0
non_striker           0
batsman_runs          0
extra_runs            0
total_runs            0
extras_type      246795
is_wicket             0
player_dismissed 247970
dismissal_kind   247970
fielder          251566
dtype: int64
```

4. Check info and shape after merging

```
merged_df.info()
```
[28] ✓ 0.1s

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 260920 entries, 0 to 260919
Data columns (total 36 columns):
 #   Column            Non-Null Count   Dtype
---  ------            --------------   -----
 0   match_id          260920 non-null  int64
 1   inning            260920 non-null  int64
 2   batting_team      260920 non-null  object
 3   bowling_team      260920 non-null  object
 4   over              260920 non-null  int64
 5   ball              260920 non-null  int64
 6   batter            260920 non-null  object
 7   bowler            260920 non-null  object
 8   non_striker       260920 non-null  object
 9   batsman_runs      260920 non-null  int64
 10  extra_runs        260920 non-null  int64
 11  total_runs        260920 non-null  int64
 12  extras_type       14125 non-null   object
 13  is_wicket         260920 non-null  int64
 14  player_dismissed  12950 non-null   object
 15  dismissal_kind    12950 non-null   object
 16  fielder           9354 non-null    object
 17  season            260920 non-null  object
 18  city              248523 non-null  object
 19  date              260920 non-null  object
...
 34  umpire1           260920 non-null  object
 35  umpire2           260920 non-null  object
dtypes: float64(3), int64(8), object(25)
```

```
print(merged_df.shape)
```
[31] ✓ 0.0s

```
(260920, 36)
```

5. Get the columns in the right datatype

```
merged_df['date'] = pd.to_datetime(merged_df['date'])
```

```
[ ] merged_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 260920 entries, 0 to 260919
Data columns (total 37 columns):
 #   Column            Non-Null Count   Dtype
---  ------            --------------   -----
 0   match_id          260920 non-null  int64
 1   inning            260920 non-null  int64
 2   batting_team      260920 non-null  object
 3   bowling_team      260920 non-null  object
 4   over              260920 non-null  int64
 5   ball              260920 non-null  int64
 6   batter            260920 non-null  object
 7   bowler            260920 non-null  object
 8   non_striker       260920 non-null  object
 9   batsman_runs      260920 non-null  int64
 10  extra_runs        260920 non-null  int64
 11  total_runs        260920 non-null  int64
 12  extras_type       14125 non-null   object
 13  is_wicket         260920 non-null  int64
 14  player_dismissed  12950 non-null   object
 15  dismissal_kind    12950 non-null   object
 16  fielder           9354 non-null    object
 17  season            260920 non-null  object
 18  city              248523 non-null  object
 19  date              260920 non-null  datetime64[ns]
```

6. Derive an index field and add it to the data set.

```
[18]  merged_df['index'] = merged_df.index
```

```
[19]  merged_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 260920 entries, 0 to 260919
Data columns (total 37 columns):
 #   Column            Non-Null Count   Dtype
---  ------            --------------   -----
 0   match_id          260920 non-null  int64
 1   inning            260920 non-null  int64
 2   batting_team      260920 non-null  object
 3   bowling_team      260920 non-null  object
 4   over              260920 non-null  int64
 5   ball              260920 non-null  int64
 6   batter            260920 non-null  object
 7   bowler            260920 non-null  object
 8   non_striker       260920 non-null  object
 9   batsman_runs      260920 non-null  int64
 10  extra_runs        260920 non-null  int64
 11  total_runs        260920 non-null  int64
 12  extras_type       14125 non-null   object
 13  is_wicket         260920 non-null  int64
 14  player_dismissed  12950 non-null   object
 15  dismissal_kind    12950 non-null   object
 16  fielder           9354 non-null    object
 17  season            260920 non-null  object
 18  city              248523 non-null  object
 19  date              260920 non-null  object
...
 35  umpire2           260920 non-null  object
 36  index             260920 non-null  int64
```

7. Identify the outliers in each column.

```
Generated code may be subject to a license | ArthurCLM/Project_OnCase
# prompt: Find outliers in each column

import pandas as pd
import numpy as np

def find_outliers_iqr(df):
    outliers = {}
    for col in df.select_dtypes(include=np.number): #Only numeric columns
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        outliers[col] = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
    return outliers


outliers_iqr = find_outliers_iqr(merged_df)

# Example: Accessing outliers for a specific column
# print(outliers_iqr['total_runs'])

for col, outlier_df in outliers_iqr.items():
  print(f"Outliers for column {col}:")
  print(outlier_df)
  print("-" * 20)
```

```
Outliers for column match_id:
Empty DataFrame
Columns: [match_id, inning, batting_team, bowling_team, over, ball, batter, bowler, non_striker, batsman_runs, extra_runs, total_runs, extras_type, is_wicket, player_dis
Index: []

[0 rows x 37 columns]
--------------------
Outliers for column inning:
        match_id  inning      batting_team           bowling_team  over  ball  \
15424     392190       4  Rajasthan Royals  Kolkata Knight Riders     0     1
15425     392190       4  Rajasthan Royals  Kolkata Knight Riders     0     2
15426     392190       4  Rajasthan Royals  Kolkata Knight Riders     0     3
15427     392190       4  Rajasthan Royals  Kolkata Knight Riders     0     4
30962     419121       4   Kings XI Punjab     Chennai Super Kings     0     1
...          ...     ...               ...                   ...   ...   ...
198500   1254077       4    Delhi Capitals    Sunrisers Hyderabad     0     2
198501   1254077       4    Delhi Capitals    Sunrisers Hyderabad     0     3
198502   1254077       4    Delhi Capitals    Sunrisers Hyderabad     0     4
198503   1254077       4    Delhi Capitals    Sunrisers Hyderabad     0     5
198504   1254077       4    Delhi Capitals    Sunrisers Hyderabad     0     6

                batter          bowler     non_striker  batsman_runs  ...  \
15424         YK Pathan      BAW Mendis  AD Mascarenhas             6  ...
15425         YK Pathan      BAW Mendis  AD Mascarenhas             2  ...
15426         YK Pathan      BAW Mendis  AD Mascarenhas             6  ...
15427         YK Pathan      BAW Mendis  AD Mascarenhas             4  ...
30962   DPMD Jayawardene  M Muralitharan    Yuvraj Singh             6  ...
...                  ...             ...             ...           ...  ...
198500          S Dhawan     Rashid Khan         RR Pant             0  ...
198501           RR Pant     Rashid Khan        S Dhawan             4  ...
198502           RR Pant     Rashid Khan        S Dhawan             0  ...
198503           RR Pant     Rashid Khan        S Dhawan             0  ...
198504          S Dhawan     Rashid Khan         RR Pant             0  ...

                winner  result result_margin  target_runs target_overs  \
15424   Rajasthan Royals     tie           NaN        151.0         20.0
15425   Rajasthan Royals     tie           NaN        151.0         20.0
15426   Rajasthan Royals     tie           NaN        151.0         20.0
15427   Rajasthan Royals     tie           NaN        151.0         20.0
```

8. What was the count of matches played in each season?

```
[26]  df2.groupby(['season'])['match_id'].agg('count')
```

```
season
2007/08    58
2009       57
2009/10    60
2011       73
2012       74
2013       76
2014       60
2015       59
2016       60
2017       59
2018       60
2019       60
2020/21    60
2021       60
2022       74
2023       74
2024       71
Name: match_id, dtype: int64
```

9. How many runs were scored in each season?

```
merged_df.groupby(['season'])['total_runs'].agg('sum')
```

season	total_runs
2007/08	17937
2009	16353
2009/10	18883
2011	21154
2012	22453
2013	22602
2014	18931
2015	18353
2016	18862
2017	18786
2018	19901
2019	19434
2020/21	19416
2021	18637
2022	24395
2023	25688
2024	25971

dtype: int64

10. Find the top 10 teams which has won the most number of matches

```
df2['winner'].value_counts().head(10)
```

winner	count
Mumbai Indians	144
Chennai Super Kings	138
Kolkata Knight Riders	131
Royal Challengers Bangalore	116
Rajasthan Royals	112
Kings XI Punjab	88
Sunrisers Hyderabad	88
Delhi Daredevils	67
Delhi Capitals	48
Deccan Chargers	29

dtype: int64

11. Find the Top 10 Players With Most Player of Match

```
[46] df2['player_of_match'].value_counts().head(10)
```

player_of_match	count
AB de Villiers	25
CH Gayle	22
RG Sharma	19
DA Warner	18
V Kohli	18
MS Dhoni	17
SR Watson	16
YK Pathan	16
RA Jadeja	16
AD Russell	15

dtype: int64

12. Find the Percentage of choosing field after winning the toss

```
[37] fieldCount = df2['match_id'][df2['toss_decision']=='field'].count()
     print(fieldCount/1095 * 100)
```

```
64.29223744292237
```

13. Find the Percentage of choosing to bat first after winning the toss

```
[36] batCount = df2['match_id'][df2['toss_decision']=='bat'].count()
     print(batCount/1095 * 100)

     35.70776255707763
```

14.Find the ratio of Bat to Field count

```
[27] batCount = df2['match_id'][df2['toss_decision']=='bat'].count()

[28] fieldCount = df2['match_id'][df2['toss_decision']=='field'].count()

[29] print(batCount/fieldCount)

     0.5553977272727273
```

15&16.Find the count of Loss vs Wins vs Ties 16. Who won the most tosses?

```
[32] df2['toss_winner'].value_counts().head(1)
```

	count
toss_winner	
Mumbai Indians	143

dtype: int64

17.Who has hit the most number of 4's?

```
[35] grouped_df.loc[4].sort_values(ascending=False).head(1)
```

	batter
batter	
S Dhawan	768

dtype: int64

18.Who has hit the most number of 6's?